Natural Language Technology

Week 06 – Information Retrieval

Today

- Quick Intro to Information Retrieval
- Indexing
- Querying
- Considering the Graph
- Vector Databases for Document Search
- Document Clustering

Information Retrieval

Information Retrieval – Is Huge!



Information Retrieval – Is Huge!

Information retrieval can be an entire course: see below for example.

Introduction to Information Retrieval: Slides

Powerpoint slides are from the Stanford CS276 class and from the Stuttgart IIR class. Latex slides are from the Stuttgart IIR class. The latex slides are in latex beamer, so you need to know/le these beamer style files.

Key: ppt = powerpoint, pdf = pdf optimized for presentation, pdf = pdf optimized for printing, src = latex source of pdf

		powerpoint				latex		
01	Boolean retrieval	ppt1	ppt2	pdf	pdf	pdf	pdf	src
02	The term vocabulary & postings lists	ppt1	ppt2	pdf	pdf	pdf	pdf	src
03	Dictionaries and tolerant retrieval	ppt1	ppt2	pdf	pdf	pdf	pdf	src
04	Index construction	ppt1	ppt2	pdf	pdf	pdf	pdf	src
05	Index compression	ppt1	ppt2	pdf	pdf	pdf	pdf	src
06	Scoring, term weighting & the vector space model	ppt1	ppt2	pdf	pdf	pdf	pdf	src
07	Computing scores in a complete search system	ppt1	ppt2	pdf	pdf	pdf	pdf	src
08	Evaluation in information retrieval	ppt1	ppt2	pdf	pdf	pdf	pdf	src
09	Relevance feedback & query expansion	ppt1	ppt2	pdf	pdf	pdf	pdf	src
10	XML retrieval	(old)	ppt2			pdf	pdf	src
11	Probabilistic information retrieval	(old)	ppt2			pdf	pdf	src
12	Language models for information retrieval	(old)	ppt2			pdf	pdf	src
13	Text classification & Naive Bayes	ppt1	ppt2	pdf	pdf	pdf	pdf	src
14	Vector space classification	ppt1	ppt2	pdf	pdf	pdf	pdf	src
15-1	Support vector machines	ppt1	ppt2	pdf	pdf	pdf	pdf	src
15-2	Learning to rank	ppt1	ppt2	pdf	pdf	pdf	pdf	src
16	Flat clustering	ppt1	ppt2	pdf	pdf	pdf	pdf	src
17	Hierarchical clustering	ppt1	ppt2	pdf	pdf	pdf	pdf	src
18	Matrix decompositions & latent semantic indexing	ppt1	ppt2	pdf	pdf	pdf	pdf	src
19	Web search basics	ppt1	ppt2	pdf	pdf	pdf	pdf	src
20	Web crawling and indexes	ppt1	ppt2	pdf	pdf	pdf	pdf	src
21	Link analysis	ppt1	ppt2	pdf	pdf	pdf	pdf	src

Slides have also been published by a number of other instructors who are using the book, e.g., by Jim Martin, Donald Patterson Min-Yen Kan, and Zhang & Helmer.

We will not aim to cover IR comprehensively. However, we will try to highlight key concepts – particularly a they relate to other key concepts in NLT.

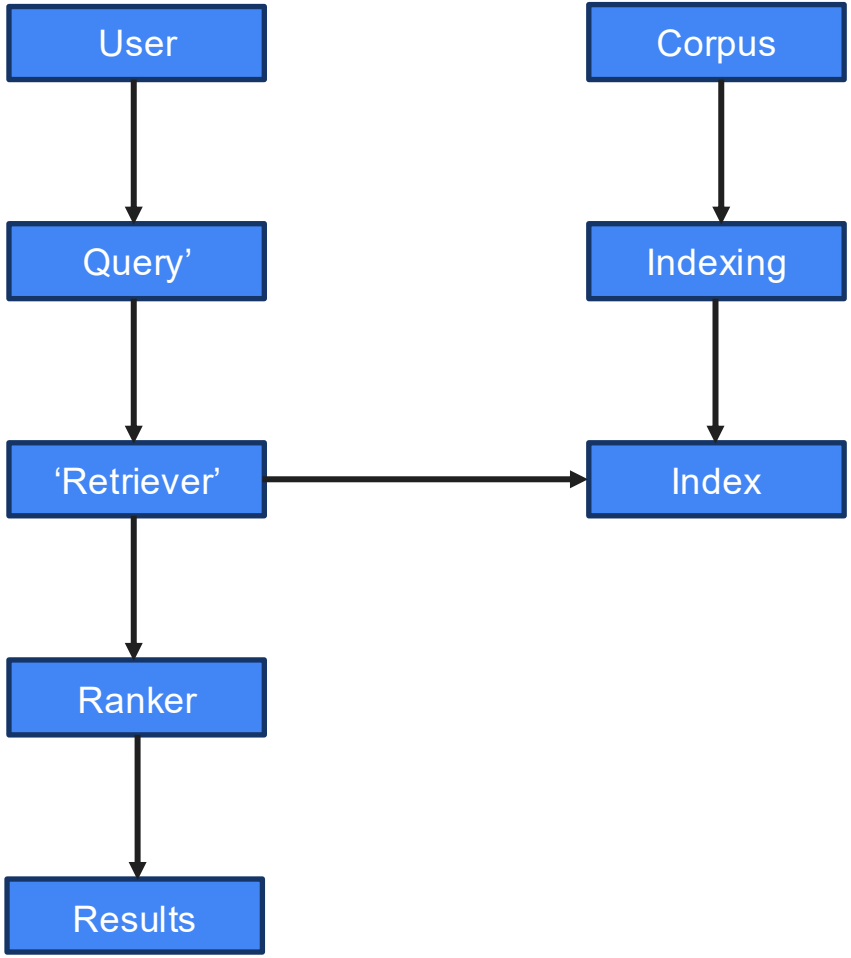
What is Information Retrieval / IR?

- **Information Retrieval is defined as the task of finding documents that meet a particular search requirement**
  - **html pages**
  - **Text documents**
  - **Media files etc**
- While IR can be very much a **multimodal problem**, in this course we are focused on the **classical text problem** of IR
- The **web** is perhaps the best-known corpus of documents that you have to search, but **local text repositories** are often still very important for enterprise solutions

Components of Classic IR System

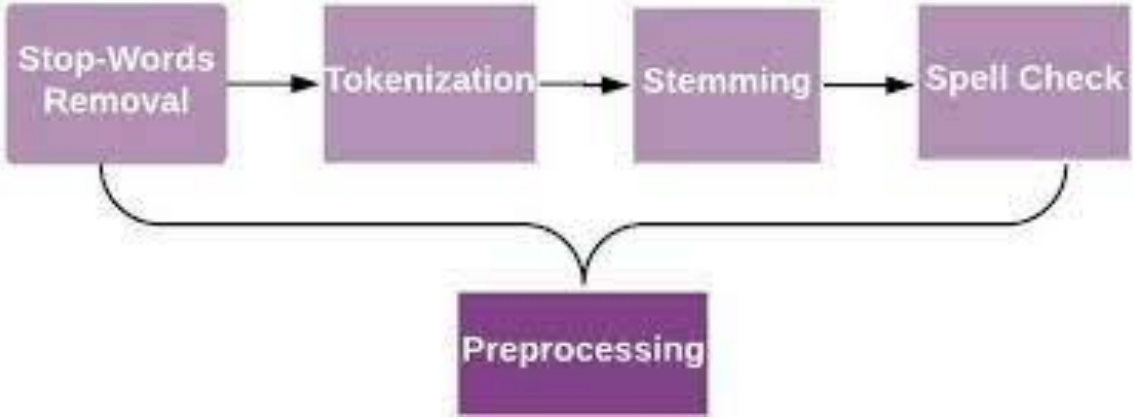
The components of an Information Retrieval system is split between the query and the indexing process which works either continually or in a periodic process over the corpus.

The query composed by the user first performs some form of flat retrieval against an index. Then either as a sperate process or a closely aligned process, we rank the matching results.



Preprocessing for Indexing

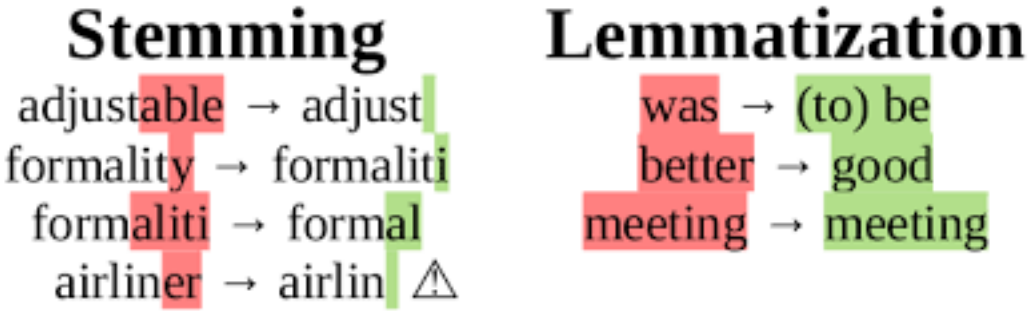
As with other text-based tasks we have looked at, a proportion of text preprocessing is often useful in improving the quality of results. The amount of preprocessing required has generally gone down over the years as embeddings-based methods (see next few weeks) have replaced more traditional TF-IDF and other 'term' based searches. Still though, we should be aware of these methods.



Stop Word Removal = Remove very frequent words seen across all documents
Tokenization = Split the document into a sequence of words
Stemming = Remove variance by reducing words to their canonical forms.

Aside: Stemming vs Lemmatization

- Stemming really means getting to the base of a given word with eg. Suffix removal rules.
- Lemmatization means converting to the ‘canonical form’ of a word and would usually require identifying POS for a given word



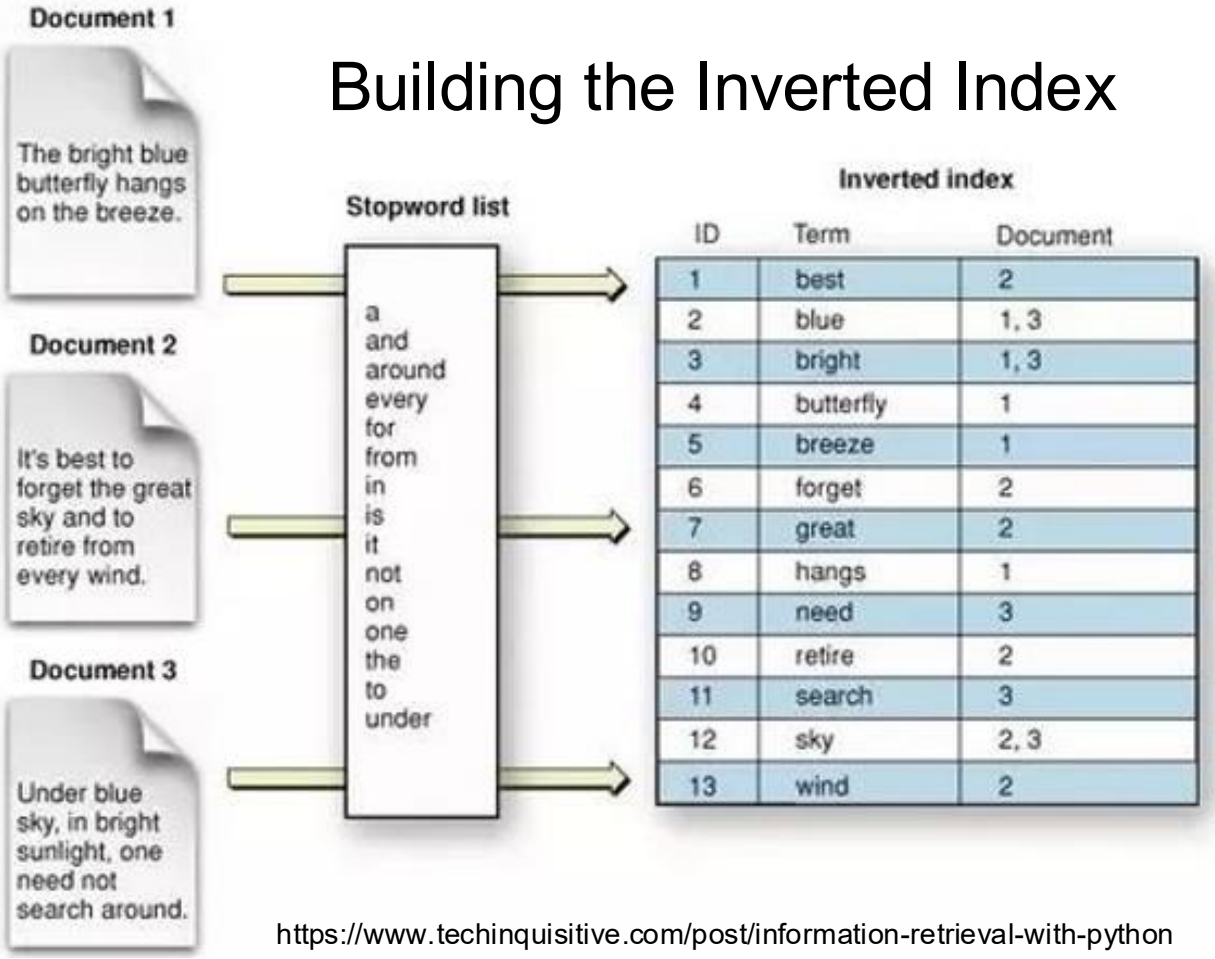
Stemming vs. Lemmatization

Attribute	Stemming	Lemmatization
Primary Mechanism	Heuristic, rule-based suffix chopping	Dictionary/morphological analysis + POS tagging
Output	Stems (may not be valid dictionary words, e.g., studi)	Valid root words (lemmas, e.g., study)
IR Trade-off	High recall, lower precision (risk of over-stemming)	High precision, lower recall (stricter matching)
Computational Cost	Extremely fast, minimal memory	Slow, requires heavy NLP pipelines & lookup tables
Classical IR Usage	Default choice for engines like Lucene/Elasticsearch	Reserved for specialized or domain-specific search

IR Process: Indexing

- Building the index has typically been the great challenge in IR.
- Rather than searching through each individual document in real time, the idea is to have some encapsulation of the data that can be stored in a more efficient index
- In classical NLP IR systems, the index is based on an **inverted index**
  - Inverted Index: For each word we have a list of documents that it occurred in.
  - This is similar to **the index in a book**

Building the Inverted Index



https://www.techinquisitive.com/post/information-retrieval-with-python

Retrieval & Ranking: Using the Inverted Index

- Take the Query
- Tokenize and convert to base terms
  - i.e., the form used in the inverted index (usually stemmed words)
- Collect info on all documents that contain matching terms
- Apply scoring / ranking logic to decide on most likely document

- What sort of logic might we apply here???

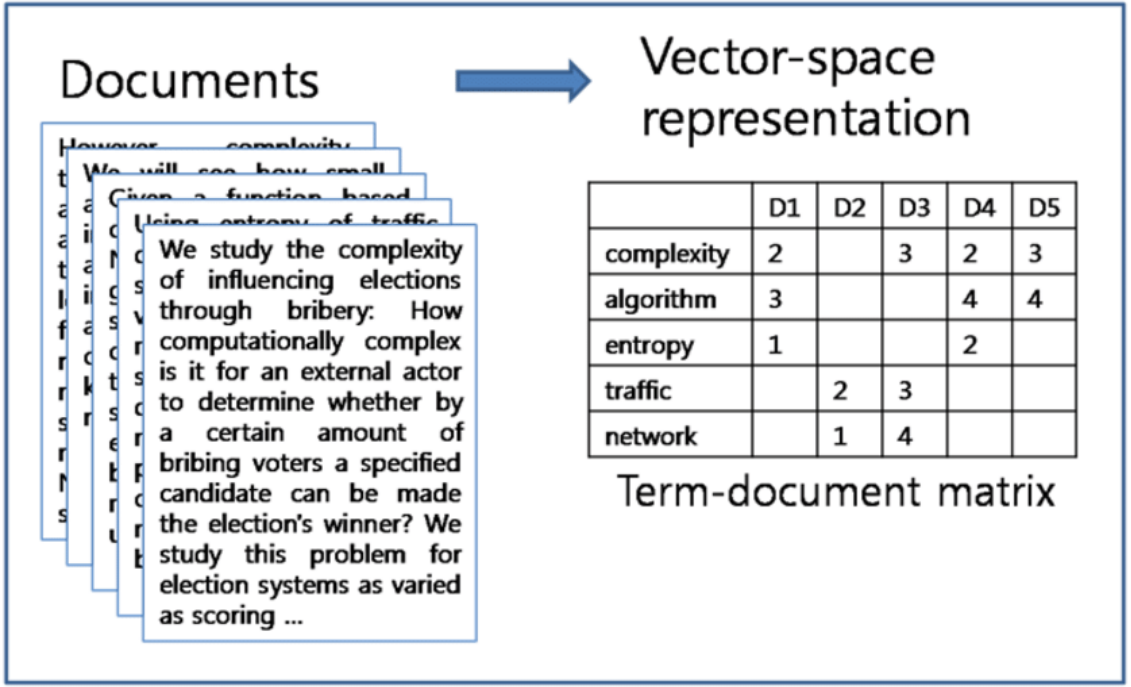
Boolean Retrieval

- **Exact Matching:** The system directly matches query terms with terms in the inverted index.
- **Boolean Operators:** Users can combine terms using logical operators like AND, OR, and NOT to specify precise search criteria.
- **Term Frequency-Inverse Document Frequency (TF-IDF):** This technique assigns weights to terms based on their frequency in documents and the entire corpus. It helps rank documents based on relevance

- At a high-level this process is very simple. In practice the algorithms to do this quickly for large indices are non-trivial.

Cosine Similarity for Document Comparison

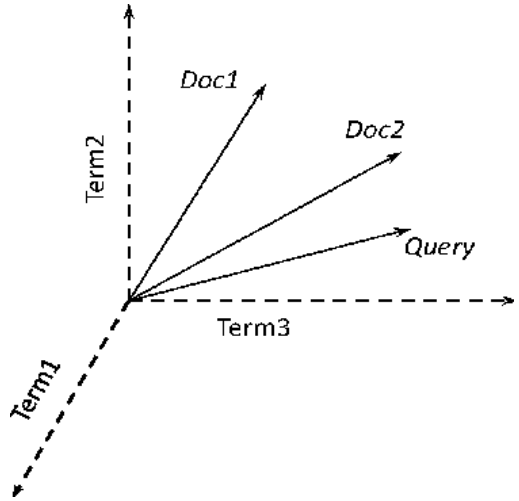
Searching based on Document Similarity



	D1	D2	D3	D4	D5
complexity	2		3	2	3
algorithm	3			4	4
entropy	1			2	
traffic		2	3		
network		1	4		

Term-document matrix

https://www.researchgate.net/figure/Figure4Documentrepresentationinthe VectorSpaceModel22_fig1_312471174



(Remember, we have seen this before. This is a Term-Freqency representation of a document. We looked at these and also the Term-Frequency – Inverse Document Frequency)

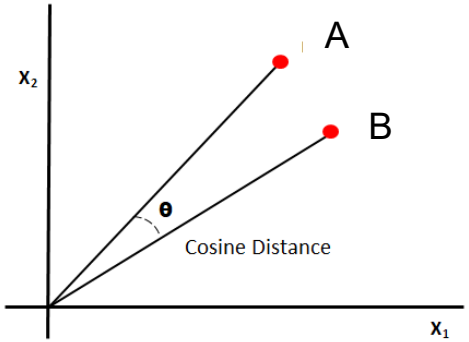
(we aren't looking at distributed vectors just yet)

Cosine Similarity

Dot product

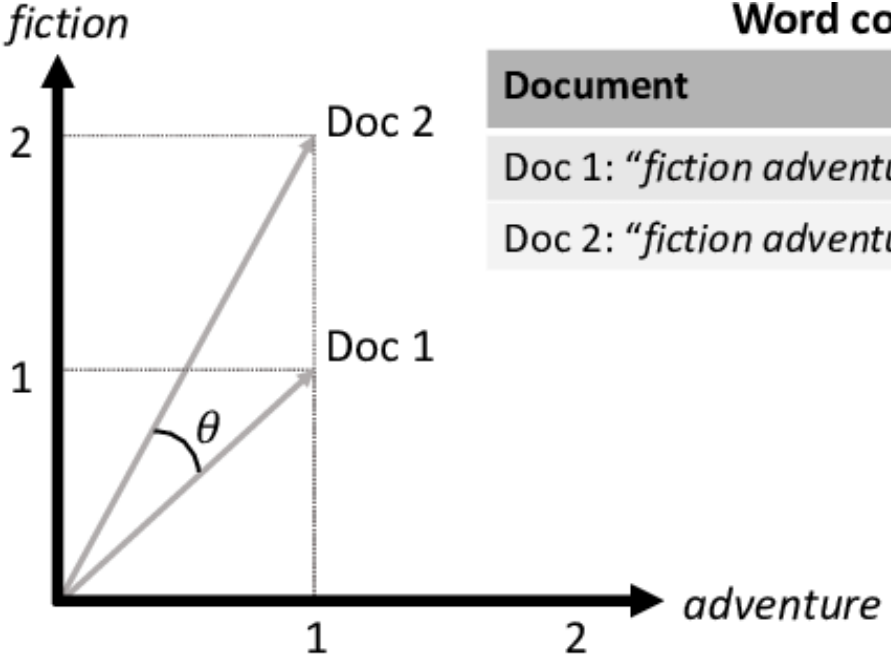
$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Vector magnitudes



The cosine between two vectors is a measure of the distance between those vectors. The division by the magnitudes of the vector actually is just normalizing to take account and neutralize the effect of the length of the vectors. In high-dimensional spaces most document vectors are not going to be pointing in the same direction (co-directional)

Running The Query – Document Similarity Measure



Word count per document

Document	*fiction*	*adventure*
Doc 1: "*fiction adventure*"	1	1
Doc 2: "*fiction adventure fiction*"	2	1

$$\cos\theta = \frac{\overrightarrow{Doc\ 1} \cdot \overrightarrow{Doc\ 2}}{\left\|\overrightarrow{Doc\ 1}\right\| \left\|\overrightarrow{Doc\ 2}\right\|}$$

$$= \frac{(1 * 2) + (1 * 1)}{\sqrt{1^2 + 1^2}\sqrt{2^2 + 1^2}}$$

$$= 0.95$$

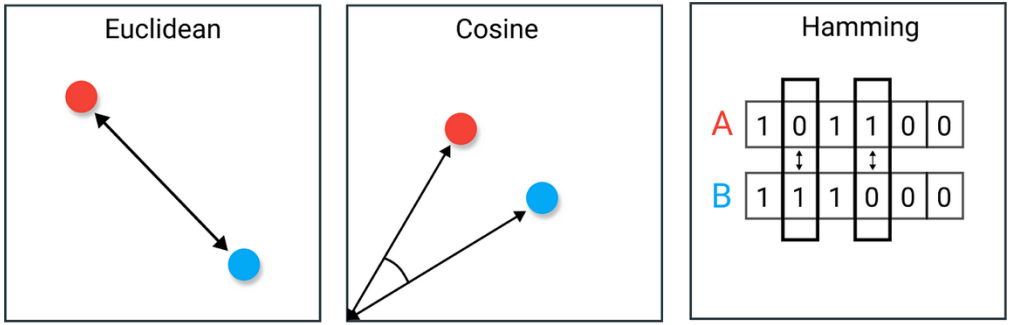
Similarity Measures

There are many similarity measures used in data science apart from Cosine Distance.

For our purposes here we don't need to know about the others. However, out of interest:

- **Euclidean distance** is the real-world measure of distance. It is also sometimes used to measure similarity – see clustering later.
- **Hamming distance and Manhattan distance** measure the number of changes that must be made to turn one vector / structure into another. Both have both had heavy influence in NLP for spelling and certain types of document

There is an excellent intro to these different measures in https://towardsdatascience.com/9-distance-measures-in-data-science-918109d069fa



From Documents to Knowledge Graphs

Beyond Flat Corpora

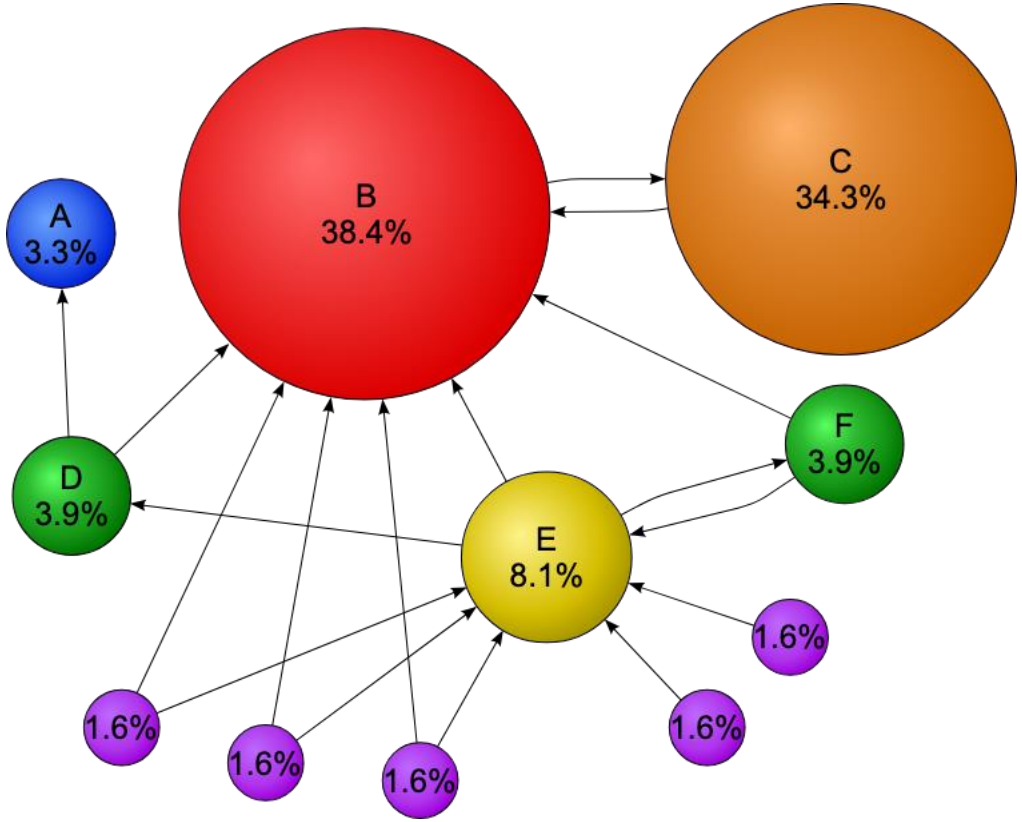
- For flat corpora of documents (company repositories) flat searches can work well
- However, when the document repository has structure, we can take advantage of that structure
- Big structured collection of documents === the Web
- The best-known web / graph search algorithm is PageRank

PageRank

Count the links going to a page and use that to generate a relative importance estimate for that website.

This information can then supplement the actual textual index results to build up an overall ranking of results.

The power of PageRank relative to the earlier generation of flat structure-based indexes for the web set Google apart from its competitors 25 years ago



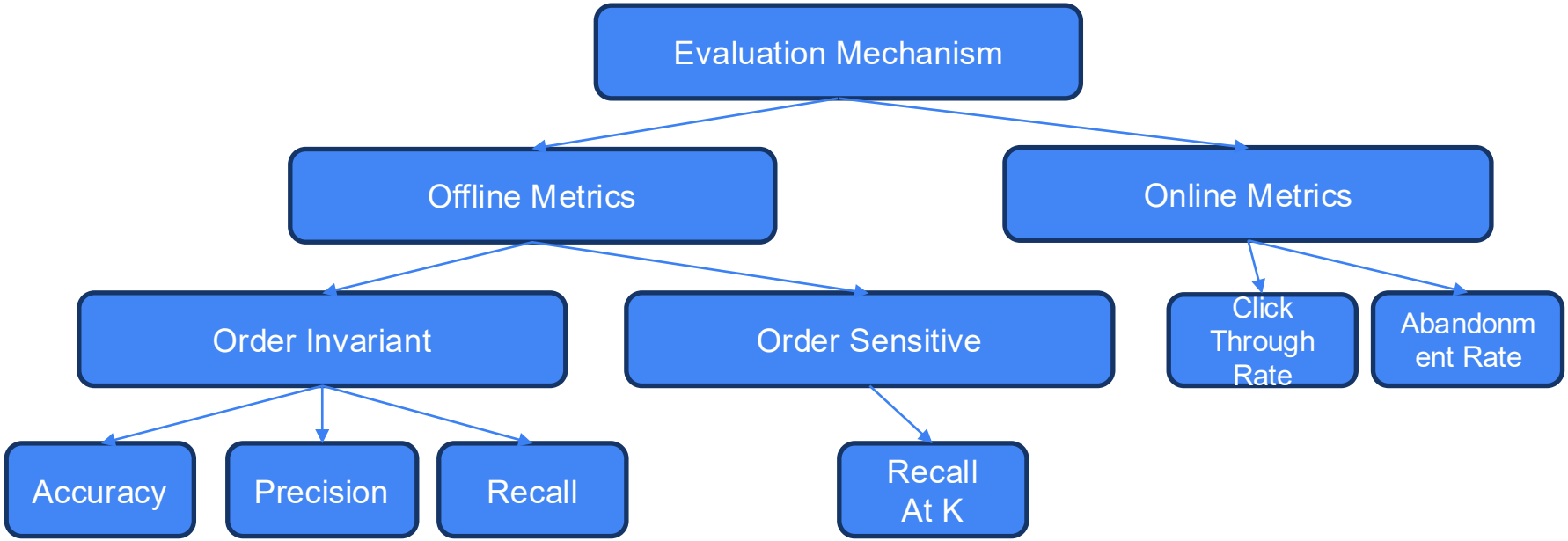
Page Rank Challenges

- Search Engine Optimization

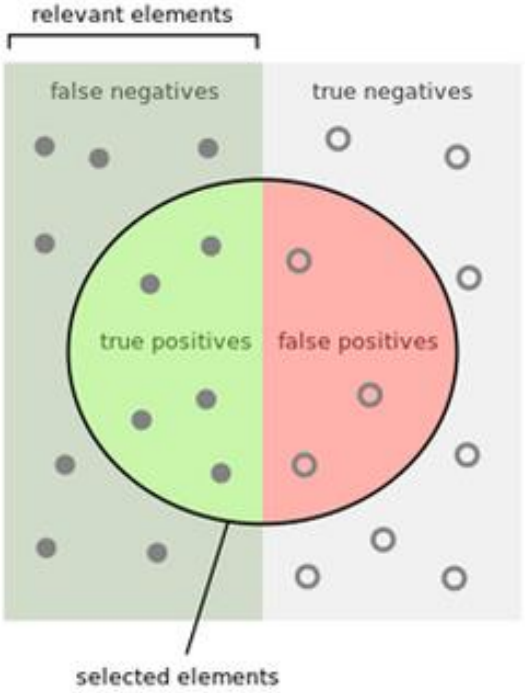
- Breaking down of web structure due to quality of search

Evaluating Information Retrieval

Evaluation Measures for Information Retrieval



Precision and Recall



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{selected elements}}$$

Ideally the red bit would be small.

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{relevant elements}}$$

Ideally the dark green area would be small. .

https://towardsdatascience.com/whats-the-deal-with-accuracy-precision-recall-and-f1-f5d8b4db1021

Recall and Precision at K

- Remember that we introduced Precision and Recall last week.
- Measures how many of the relevant items are present in top K out of all the relevant items, where K is the number of recommendations generated for a user
- Gives a measure of "out of K" items recommended to a user and how many are relevant, where K is the number of recommendations generated for a user..

$$precision@k = \frac{\text{number of recommended relevant items among top k}}{\text{number of recommended items k}}$$

$$recall@k = \frac{\text{number of recommended relevant items among top k}}{\text{number of all relevant items in the system}}$$

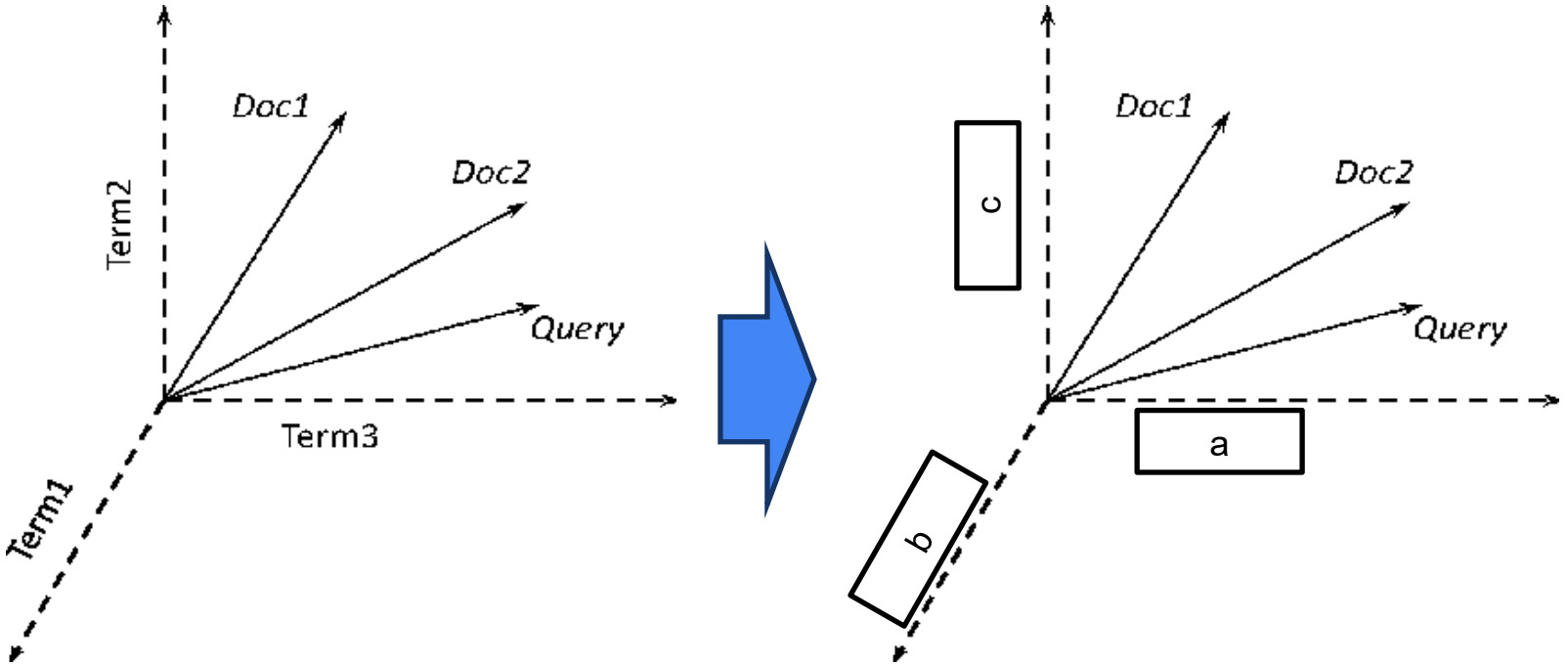
- See the great article below for more detail if needed (and many more metrics!)
- https://towardsdatascience.com/evaluation-metrics-for-recommendation-systems-an-overview-71290690ecba#:~:text=Recall%20%40K,10%20movies%20for%20every%20user.

Worked Example

- Imagine a page recommendation system with a corpus of 10,000 pages. For a specific query, the system returns the following 10 pages:
  - Page1, Page2, Page3, Page4, Page5, Page6, Page7, Page8, Page9, Page10
- The user believes Page2, Page4 and Page8 are relevant.
- We can calculate Precision@k and Recall@k for the first 5 items (or any other k)
  - Precision@5 = 2/5 = 40%
  - Recall@5 = 2/3 = 66%

(Distributed) Vector Search

Start with Classical Document Vectors but Generalize

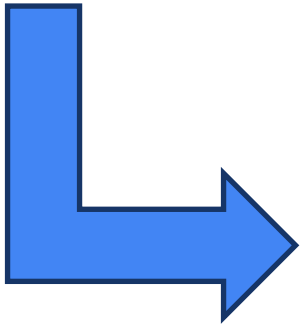


Document Index search as used in classical information retrieval

Distributed Representation. The vectors a, b, c don't have to correspond to any single concept, but if you like think about them as things like 'goodness' 'Irishness' etc.

	...	brown	...	dogs	...	fox	...	the	...
the								1	
quick									
brown		1							
fox						1			
jumped									
over									
the								1	
lazy									
dogs				1					

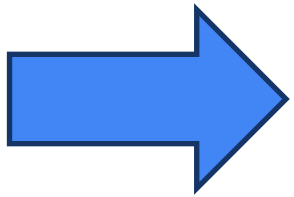
Remember we have seen something like this before when we were talking about representations of text



	f1	f2	f3	f4	f5	f6	f7	f8	...
the	0.11	0.18	0.85	0.5	0.5	0.33	0.444	0.99	
quick	0.55	0.48	0.22	0.48	0.58	0.73	0.44	0.212	
brown	0.48	0.22	0.48	0.812	0.4	0.33	0.19	0.57	
fox	0.812	0.33	0.444	0.99	00.73	1	0.04	0.02	
jumped	0.99	0.73	0.44	0.212	0.18	0.85	0.48	0.22	
over	0.212	0.001	0.3	0.129	0.48	0.22	0.48	0.373	
the	0.11	0.18	0.85	0.5	0.5	0.33	0.444	0.99	
lazy	0.44	0.22	0.901	0.33	0.44	0.18	0.85	0.5	
dogs	0.33	0.49	0.828	0.73	0.44	0.48	0.22	0.48	

Documents: Document or Sentence Embeddings

The quick brown fox jumped over the lazy dogs

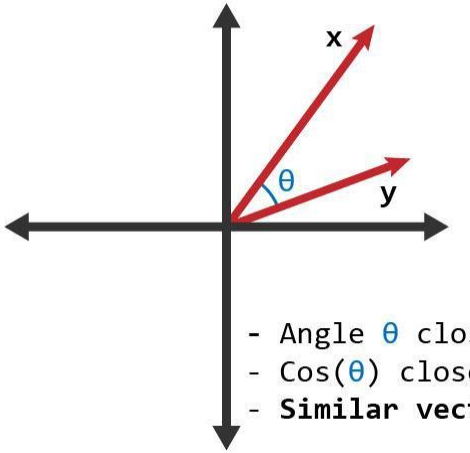


Embedding generation mechanism

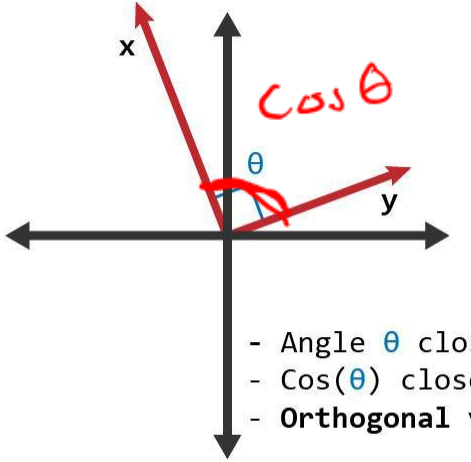
For example, universal sentence embeddings

```
tensor([[0.1111, 0.6667],
        [0.3333, 0.3333],
        [0.0000, 1.0000],
        [0.3333, 0.4444],
        [0.2222, 0.2222],
        [0.8889, 0.2222],
        [0.0000, 0.4444],
        [1.0000, 0.4444],
        [0.3333, 0.7778],
        [0.7778, 0.8889]], dtype=torch.float64)
```

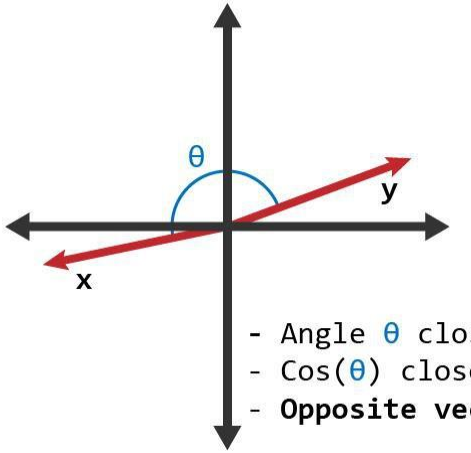
For Similarity, we still use Cosine



- Angle θ close to 0
- Cos(θ) close to 1
- **Similar vectors**

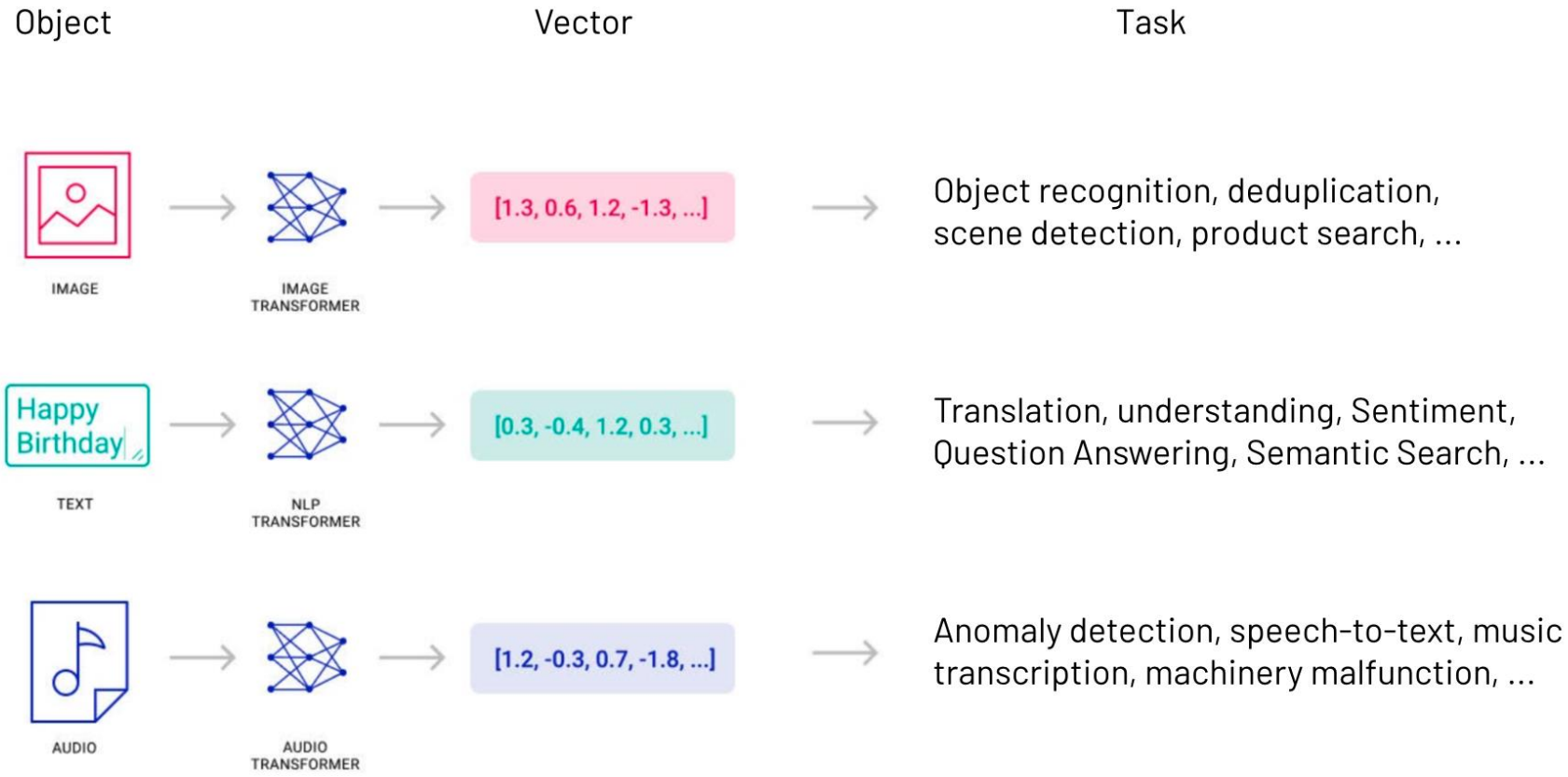


- Angle θ close to 90
- Cos(θ) close to 0
- **Orthogonal vectors**



- Angle θ close to 180
- Cos(θ) close to -1
- **Opposite vectors**

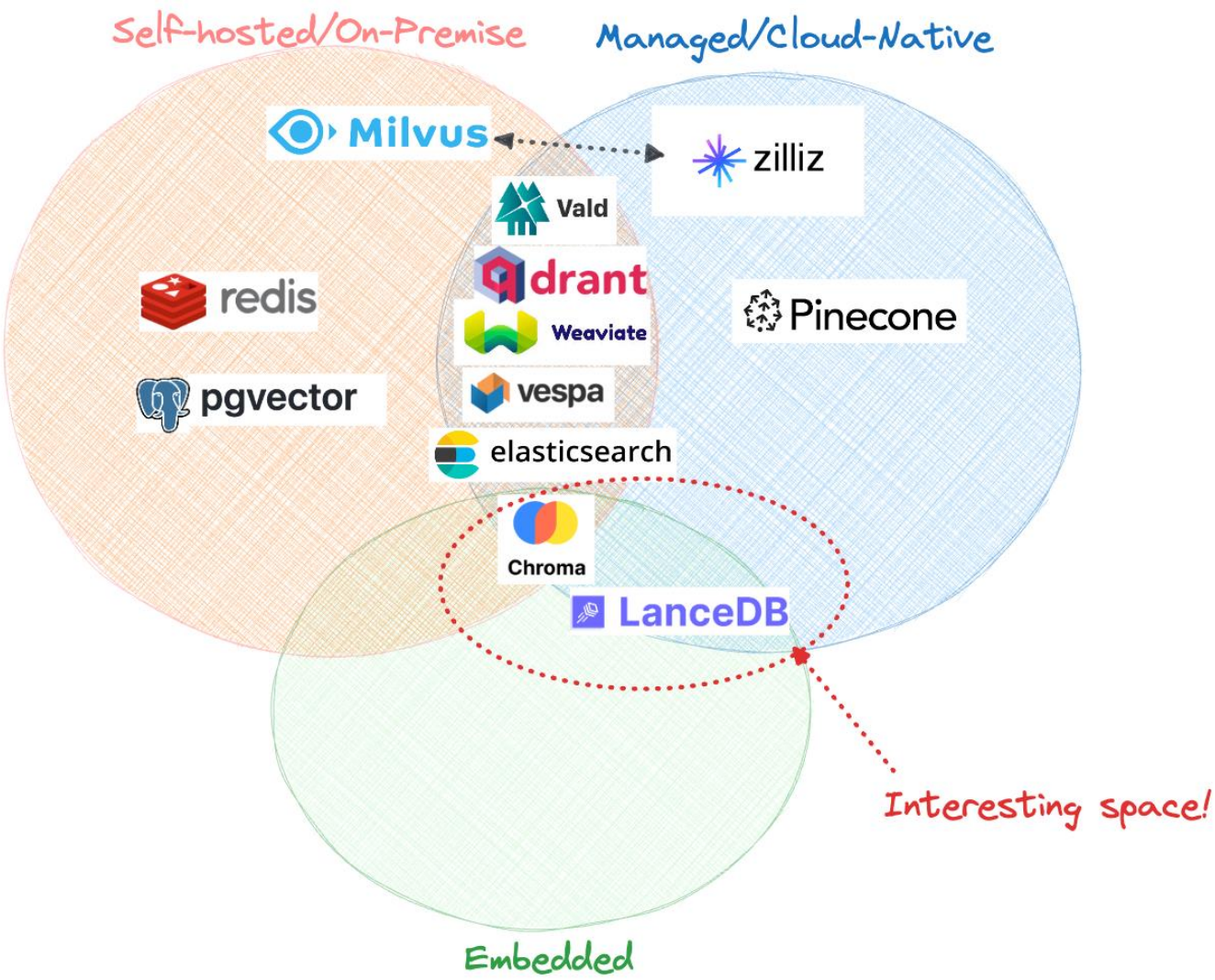
From Information Retrieval to Vector Databases



Vector Database Ecosystem



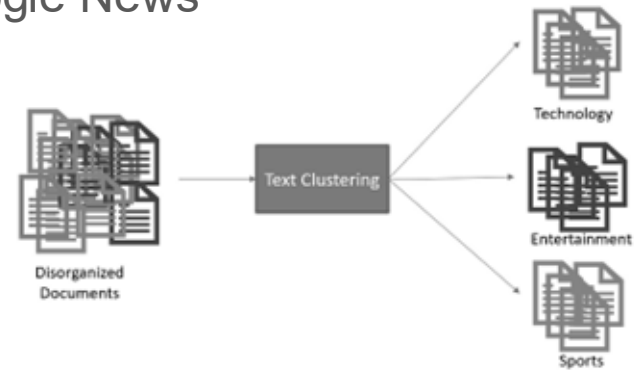
https://www.linkedin.com/pulse/choosing-vector-database-your-gen-ai-stack-abhinav-srivastava


Self-hosted/On-Premise
Managed/Cloud-Native
Milvus
zilliz
Vald
drant
redis
Weaviate
Pinecone
pgvector
vespa
elasticsearch
Chroma
LanceDB
Interesting space!
Embedded

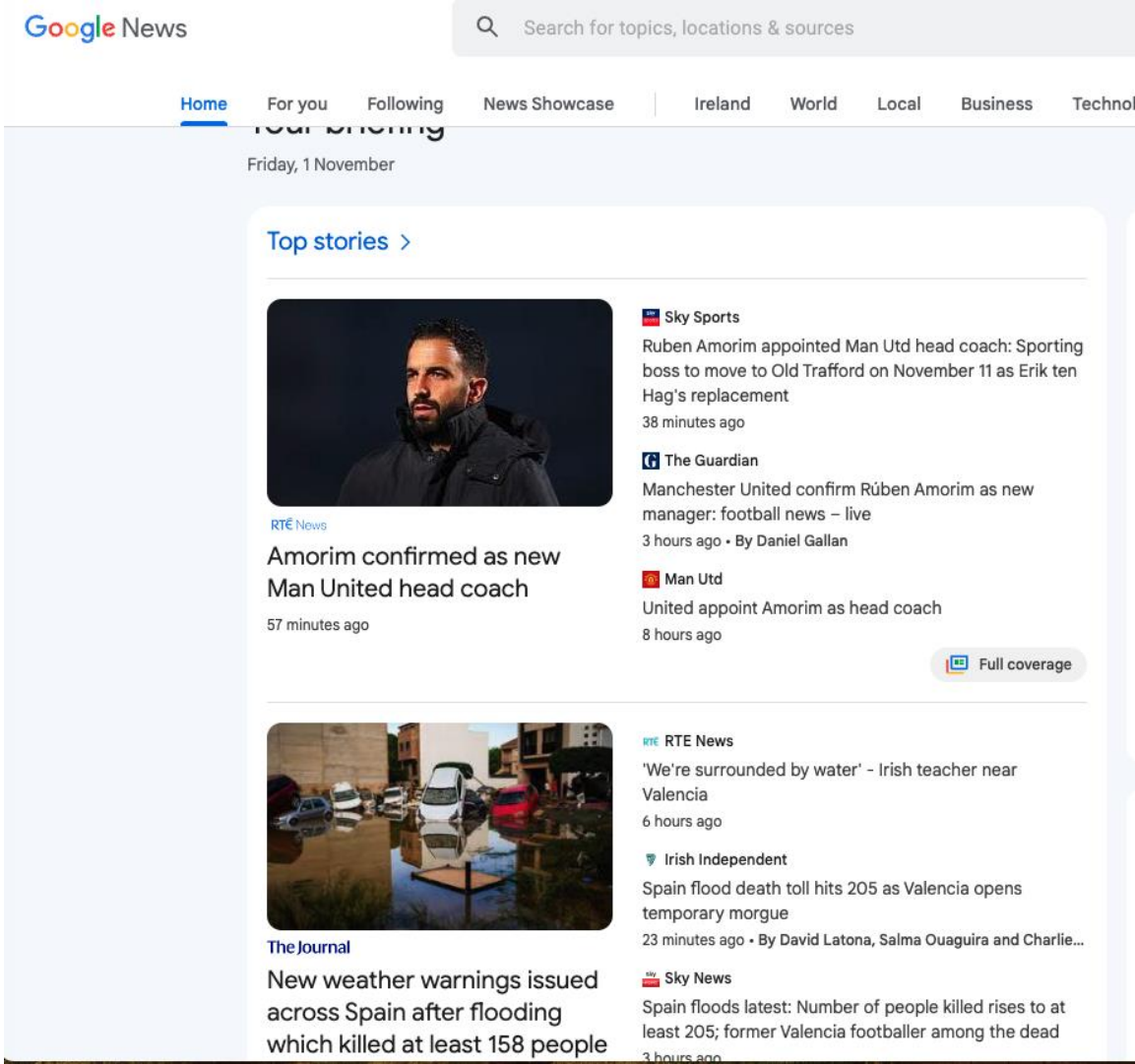
Document Clustering

What is (Document) Clustering?

- Clustering is the process of automatically sorting items into sets with similar features. Clustering is used widely when we want to explore the data which is maybe not labelled already.
- We can cluster documents into groups to help some algorithms, but also to help people get a better view on topics. Clustering doesn’t require someone to add the labels manually.
- Example of document clustering in process: Google News

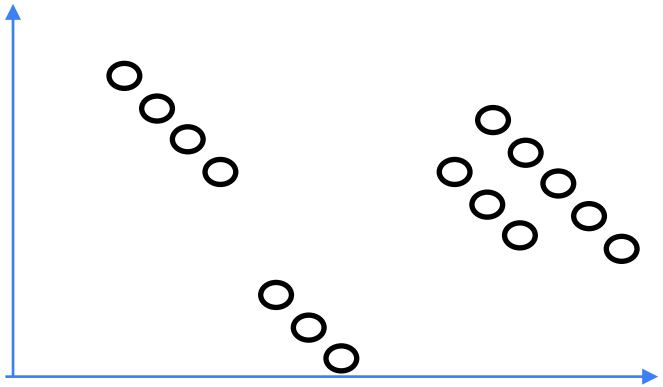


Document Clusters identified by Google News



A Basic Approach: K-Means Clustering

- K-Means clustering is an unsupervised clustering algorithm which takes as a parameter n, the number of clusters expected. It then returns to us n points in the data space. Each of these points is interpreted as the centroid of a stable cluster in that space.

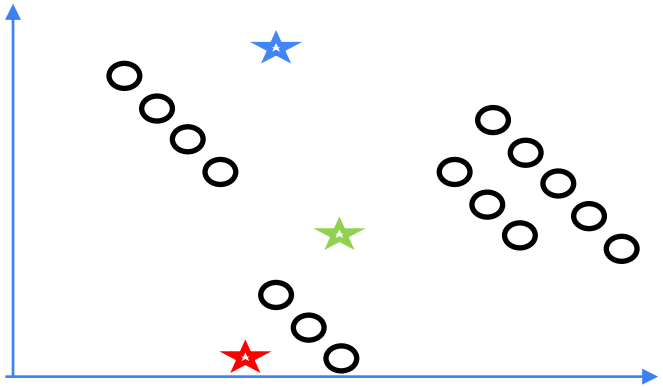


https://neptune.ai/blog/k-means-clustering

A Basic Approach: K-Means Clustering

- K-Means clustering is an unsupervised clustering algorithm which takes as a parameter n, the number of clusters expected. It then returns to us n points in the data space. Each of these points is interpreted as the centroid of a stable cluster in that space.

Step 1: Randomly initialize n centroids

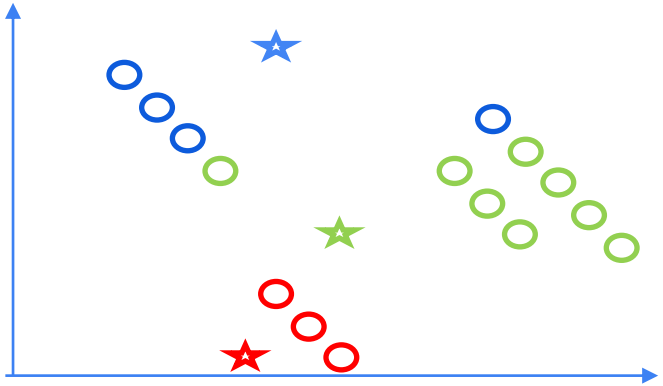


https://neptune.ai/blog/k-means-clustering

A Basic Approach: K-Means Clustering

- K-Means clustering is an unsupervised clustering algorithm which takes as a parameter n, the number of clusters expected. It then returns to us n points in the data space. Each of these points is interpreted as the centroid of a stable cluster in that space.

Step 1: Randomly initialize n centroids
Step 2: Assign each object to its closest centroid



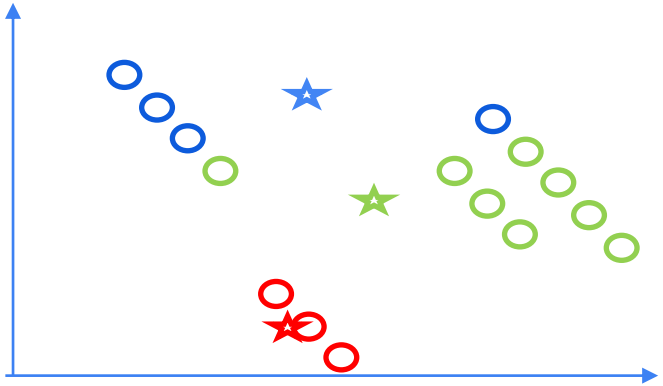
$$d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

https://neptune.ai/blog/k-means-clustering

A Basic Approach: K-Means Clustering

- K-Means clustering is an unsupervised clustering algorithm which takes as a parameter n, the number of clusters expected. It then returns to us n points in the data space. Each of these points is interpreted as the centroid of a stable cluster in that space.

Step 1: Randomly initialize n centroids
Step 2: Assign each object to its closest centroid
Step 3: Move centroids to be in the middle of their datapoints



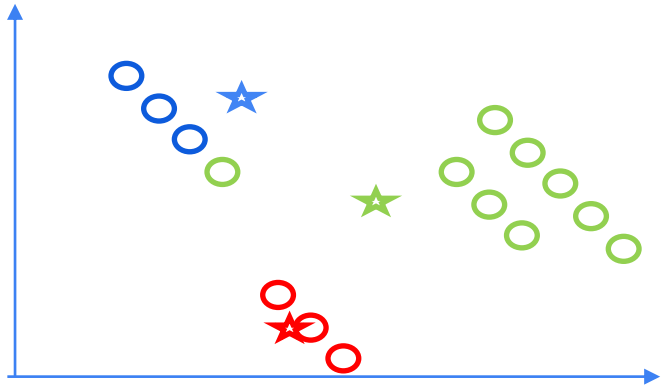
$$Ci = \frac{1}{|Ni|} \sum xi$$

https://neptune.ai/blog/k-means-clustering

A Basic Approach: K-Means Clustering

- K-Means clustering is an unsupervised clustering algorithm which takes as a parameter n, the number of clusters expected. It then returns to us n points in the data space. Each of these points is interpreted as the centroid of a stable cluster in that space.

Step 1: Randomly initialize n centroids
Step 2: Assign each object to its closest centroid
Step 3: Move centroids to be in the middle of their datapoints
Step 4: Update assignments.

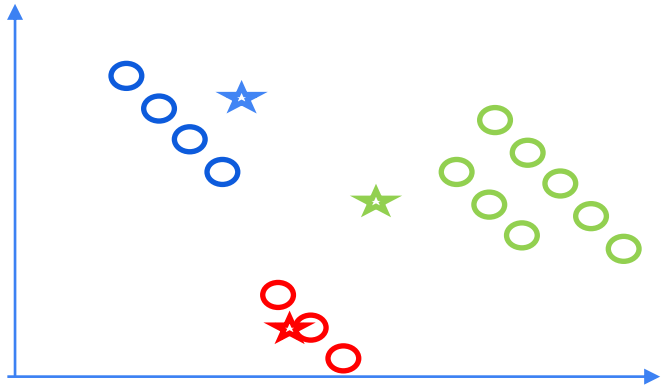


https://neptune.ai/blog/k-means-clustering

A Basic Approach: K-Means Clustering

- K-Means clustering is an unsupervised clustering algorithm which takes as a parameter n, the number of clusters expected. It then returns to us n points in the data space. Each of these points is interpreted as the centroid of a stable cluster in that space.

Step 1: Randomly initialize n centroids
Step 2: Assign each object to its closest centroid
Step 3: Move centroids to be in the middle of their datapoints
Step 4: Update assignments. Repeat steps 3 and 4 until convergence



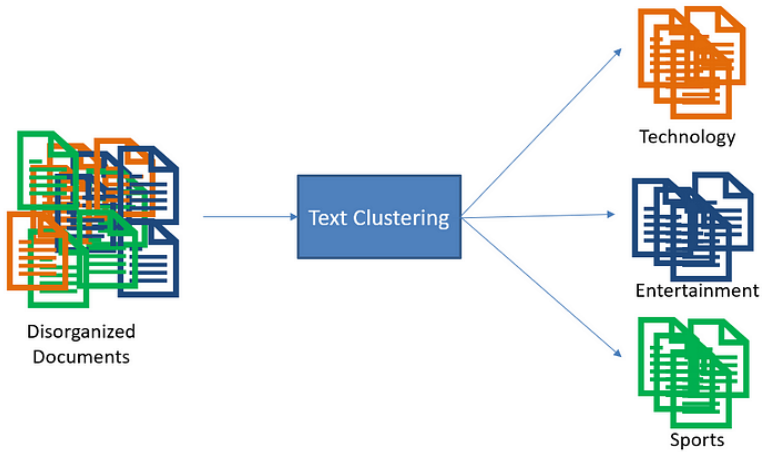
https://neptune.ai/blog/k-means-clustering

On K-Means Clustering

- While the algorithm is guaranteed to stop, you aren't guaranteed to get the global optimium set of clusters
  - Running again with a different set of initial candidates could give a better result
- K-Means requires a guess of the number of clusters which is not always easy
  - Unless you have a good knowledge of how many clusters there should be, trying different values of n will result in different quality clusters.
  - Consider the example in the last slides. What would the clusters look like if there had been maybe 4 or 2 clusters suggested?

K-Means Clustering for Documents

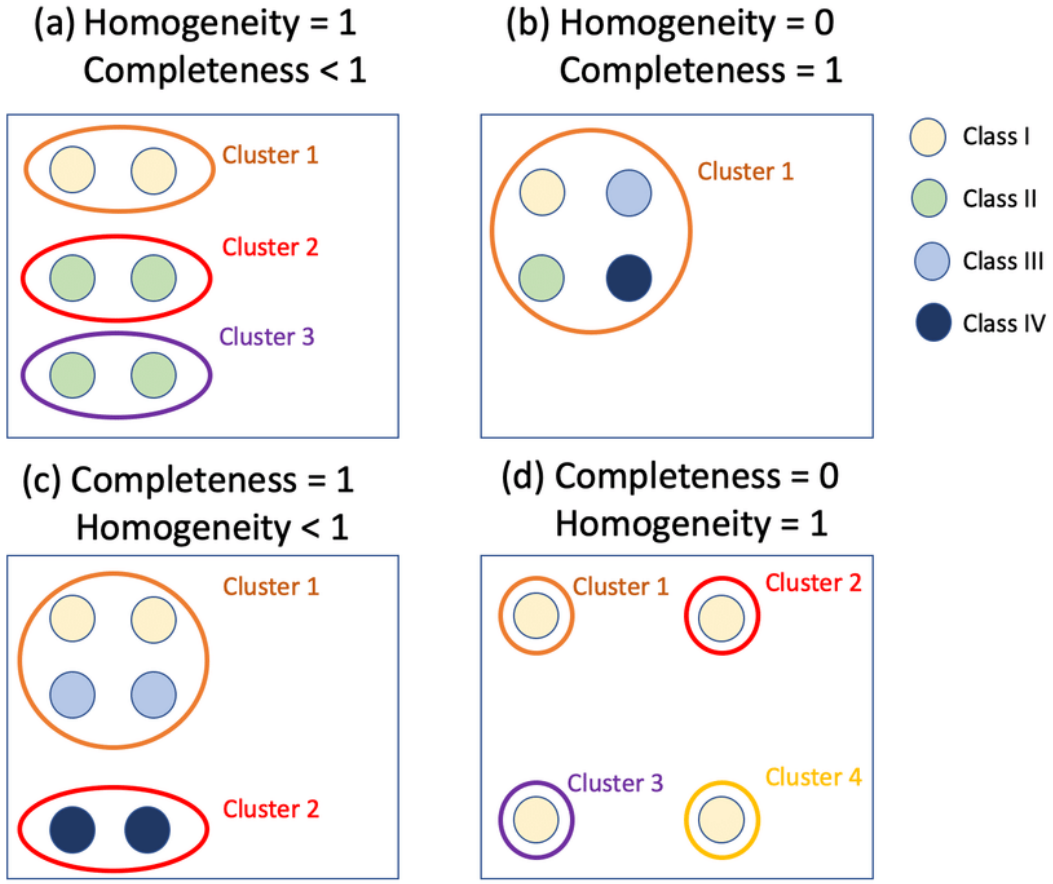
- We can directly use K-Means to cluster documents.
- Importantly, we can do this with different types of representations.
- Clustering based on TF-IDF might seem like the most transparent, but clustering based on document embeddings will usually provide better results
  - Why?



https://medium.com/@danielafrimi/text-clustering-using-nlp-techniques-c2e6b08b6e95

Evaluating Clusters I

- **Extrinsic** Measures - If we have access to the 'correct clusters'
- **Homogeneity**: Each cluster contains only members of a single class (somewhat like "*precision*")
- **Completeness**: All members of a given class are assigned to the same cluster (somewhat like "*recall*")

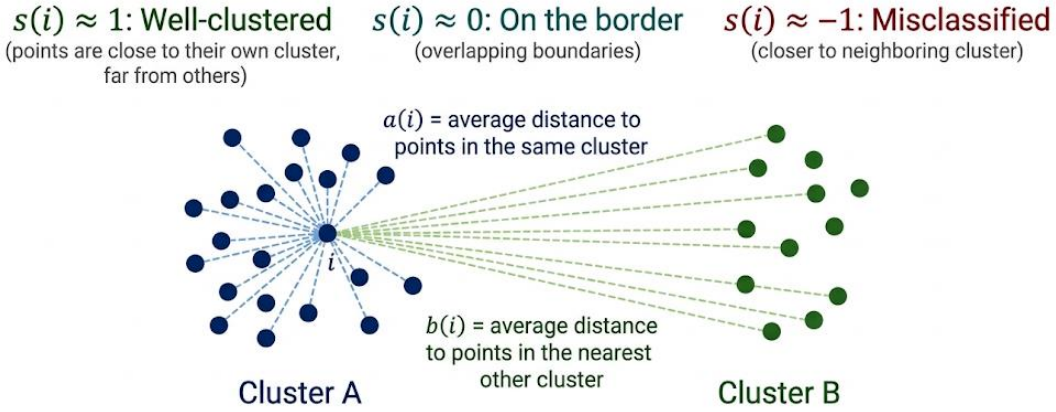


Evaluating Clusters II

- Intrinsic Measures - If we don't have access to the true labels.
  - Measures how similar a data point is to its own cluster compared to other clusters.
- This is the basic principle of the Silhouette Algorithm

The silhouette algorithm measures how well a data point fits within its assigned cluster compared to the nearest neighboring cluster by calculating a score from -1 to +1 based on the difference between its average intra-cluster distance and its average nearest-cluster distance.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$



Lab Work

K Means Clustering for Documents

- See Lab